

Apuntes

Requisitos de finalización

Acceso a Bases de Datos Relacionales con Python

Índice

1. [Conceptos Fundamentales](#)
 2. [Instalación y Configuración](#)
 3. [Conexiones Básicas](#)
 4. [SQLite](#)
 5. [MySQL/MariaDB](#)
 6. [PostgreSQL](#)
 7. [ORM con SQLAlchemy](#)
 8. [Buenas Prácticas](#)
 9. [Recursos y Referencias](#)
-

Conceptos Fundamentales

¿Qué es una Base de Datos Relacional?

Una base de datos relacional organiza los datos en **tablas** (entidades) con **filas** (registros) y **columnas** (atributos). Las relaciones entre tablas se establecen mediante **claves foráneas**.

CRUD Operations

- **Create (Crear):** INSERT
- **Read (Leer):** SELECT
- **Update (Actualizar):** UPDATE
- **Delete (Eliminar):** DELETE

Conexión a BD

El proceso típico es:

1. Importar el módulo/librería
2. Crear una conexión con la BD
3. Crear un cursor (ejecutor de comandos)
4. Ejecutar las operaciones SQL
5. Confirmar cambios (commit)
6. Cerrar la conexión

Instalación y Configuración

Herramientas Necesarias

```
# SQLite (incluido en Python, sin instalación)
python --version

# MySQL
pip install mysql-connector-python
# O alternativa con PyMySQL
pip install pymysql

# PostgreSQL
pip install psycopg2-binary

# SQLAlchemy (ORM)
pip install sqlalchemy

# Herramientas adicionales útiles
pip install python-dotenv # Para variables de entorno
pip install pandas        # Para análisis de datos
```

Conexiones Básicas

Estructura General de una Conexión

```
import sqlite3
# o: import mysql.connector, psycopg2, etc.

# 1. Crear conexión
conexion = sqlite3.connect('base_datos.db')

# 2. Crear cursor
cursor = conexion.cursor()

# 3. Ejecutar operaciones
cursor.execute("SELECT * FROM usuarios")
resultados = cursor.fetchall()

# 4. Confirmar cambios (importante para INSERT, UPDATE, DELETE)
conexion.commit()

# 5. Cerrar conexión
cursor.close()
conexion.close()
```

Usar Context Manager (Recomendado)

```
import sqlite3

# Cierra automáticamente la conexión
with sqlite3.connect('base_datos.db') as conexion:
    cursor = conexion.cursor()
    cursor.execute("SELECT * FROM usuarios")
    for fila in cursor.fetchall():
```

```
        print(fila)
    # commit() se hace automáticamente al salir
```

SQLite

SQLite es la opción ideal para proyectos pequeños, pruebas y aplicaciones desktop. Los datos se almacenan en un archivo `.db`.

Crear Tabla

```
import sqlite3

conexion = sqlite3.connect('tienda.db')
cursor = conexion.cursor()

# Crear tabla de usuarios
cursor.execute('''
    CREATE TABLE IF NOT EXISTS usuarios (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL,
        edad INTEGER,
        fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP
    )
''')

conexion.commit()
cursor.close()
conexion.close()
```

INSERT (Crear Registros)

```
import sqlite3

conexion = sqlite3.connect('tienda.db')
cursor = conexion.cursor()

# Un registro
cursor.execute('''
    INSERT INTO usuarios (nombre, email, edad)
    VALUES (?, ?, ?)
''', ('Juan', 'juan@email.com', 28))

# Múltiples registros
datos = [
    ('María', 'maria@email.com', 25),
    ('Carlos', 'carlos@email.com', 32),
    ('Ana', 'ana@email.com', 29)
]
cursor.executemany('''
    INSERT INTO usuarios (nombre, email, edad)
    VALUES (?, ?, ?)
''', datos)

conexion.commit()
print(f"Registros insertados: {cursor.rowcount}")
cursor.close()
conexion.close()
```

SELECT (Leer Datos)

```
import sqlite3

conexion = sqlite3.connect('tienda.db')
cursor = conexion.cursor()

# Obtener todos los registros
cursor.execute("SELECT * FROM usuarios")
todos = cursor.fetchall()
for fila in todos:
    print(fila)

# Obtener un registro
cursor.execute("SELECT * FROM usuarios WHERE id = ?", (1,))
un_registro = cursor.fetchone()
print(un_registro)

# Obtener múltiples con límite
cursor.execute("SELECT * FROM usuarios LIMIT 2")
algunos = cursor.fetchmany(2)
print(algunos)

cursor.close()
conexion.close()
```

UPDATE (Actualizar Registros)

```
cursor.execute('''
    UPDATE usuarios
    SET edad = ?, email = ?
    WHERE id = ?
''', (30, 'nuevo@email.com', 1))

conexion.commit()
print(f"Registros actualizados: {cursor.rowcount}")
```

DELETE (Eliminar Registros)

```
cursor.execute('DELETE FROM usuarios WHERE id = ?', (1,))
conexion.commit()
print(f"Registros eliminados: {cursor.rowcount}")
```

Row Factory (Resultados como Diccionarios)

```
import sqlite3

conexion = sqlite3.connect('tienda.db')
conexion.row_factory = sqlite3.Row # Importante
cursor = conexion.cursor()

cursor.execute("SELECT * FROM usuarios")
for fila in cursor.fetchall():
    print(fila['nombre'], fila['email'])

cursor.close()
conexion.close()
```

MySQL/MariaDB

Para bases de datos en servidor, más potentes y escalables.

Instalación y Configuración

```
pip install mysql-connector-python
```

Conexión Básica

```
import mysql.connector
from mysql.connector import Error

try:
    conexion = mysql.connector.connect(
        host='localhost',      # o IP del servidor
        user='root',          # usuario MySQL
        password='tu_password',
        database='tienda'
    )

    if conexion.is_connected():
        db_info = conexion.get_server_info()
        print(f"Conectado a MySQL Server versión {db_info}")

        cursor = conexion.cursor()
        cursor.execute("SELECT DATABASE();")
        registro = cursor.fetchone()
        print(f"Base de datos seleccionada: {registro}")

except Error as e:
    print(f"Error al conectar: {e}")

finally:
    if conexion.is_connected():
        cursor.close()
        conexion.close()
    print("Conexión cerrada")
```

Crear Tabla en MySQL

```
import mysql.connector

conexion = mysql.connector.connect(
    host='localhost',
    user='root',
    password='password',
    database='tienda'
)

cursor = conexion.cursor()

crear_tabla = '''
CREATE TABLE IF NOT EXISTS productos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio DECIMAL(10, 2) NOT NULL,
    stock INT DEFAULT 0,
    descripcion TEXT,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
```

```

'''

cursor.execute(crear_tabla)
conexion.commit()
print("Tabla creada exitosamente")

cursor.close()
conexion.close()

```

CRUD en MySQL

```

import mysql.connector

def conectar():
    return mysql.connector.connect(
        host='localhost',
        user='root',
        password='password',
        database='tienda'
    )

# CREATE
def crear_producto(nombre, precio, stock):
    conexion = conectar()
    cursor = conexion.cursor()

    sql = "INSERT INTO productos (nombre, precio, stock) VALUES (%s, %s, %s)"
    valores = (nombre, precio, stock)

    cursor.execute(sql, valores)
    conexion.commit()
    print(f"Producto {nombre} creado con ID: {cursor.lastrowid}")

    cursor.close()
    conexion.close()

# READ
def obtener_productos():
    conexion = conectar()
    cursor = conexion.cursor(dictionary=True) # Retorna diccionarios

    cursor.execute("SELECT * FROM productos")
    productos = cursor.fetchall()

    cursor.close()
    conexion.close()
    return productos

# UPDATE
def actualizar_precio(producto_id, nuevo_precio):
    conexion = conectar()
    cursor = conexion.cursor()

    sql = "UPDATE productos SET precio = %s WHERE id = %s"
    cursor.execute(sql, (nuevo_precio, producto_id))
    conexion.commit()

    print(f"Registros actualizados: {cursor.rowcount}")
    cursor.close()
    conexion.close()

# DELETE
def eliminar_producto(producto_id):

```

```

conexion = conectar()
cursor = conexion.cursor()

cursor.execute("DELETE FROM productos WHERE id = %s", (producto_id,))
conexion.commit()

print(f"Registros eliminados: {cursor.rowcount}")
cursor.close()
conexion.close()

# Uso
crear_producto('Laptop', 999.99, 5)
crear_producto('Mouse', 25.50, 50)

productos = obtener_productos()
for p in productos:
    print(p)

actualizar_precio(1, 899.99)

```

PostgreSQL

Potente BD relacional de código abierto, ideal para aplicaciones empresariales.

Instalación

```
pip install psycopg2-binary
```

Conexión Básica

```

import psycopg2
from psycopg2 import Error

try:
    conexion = psycopg2.connect(
        host='localhost',
        database='tienda',
        user='postgres',
        password='password',
        port='5432'
    )

    cursor = conexion.cursor()
    cursor.execute("SELECT version();")
    db_version = cursor.fetchone()
    print(f"Conectado a PostgreSQL: {db_version}")

except Error as e:
    print(f"Error: {e}")

finally:
    if conexion:
        cursor.close()
        conexion.close()

```

CRUD en PostgreSQL

```
import psycopg2
```

```

from psycopg2 import Error

def conectar():
    return psycopg2.connect(
        host='localhost',
        database='tienda',
        user='postgres',
        password='password'
    )

def crear_tabla():
    conexion = conectar()
    cursor = conexion.cursor()

    sql = '''
        CREATE TABLE IF NOT EXISTS clientes (
            id SERIAL PRIMARY KEY,
            nombre VARCHAR(100) NOT NULL,
            email VARCHAR(100) UNIQUE NOT NULL,
            telefono VARCHAR(20),
            fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    '''

    cursor.execute(sql)
    conexion.commit()
    print("Tabla creada")

    cursor.close()
    conexion.close()

def insertar_cliente(nombre, email, telefono):
    conexion = conectar()
    cursor = conexion.cursor()

    sql = "INSERT INTO clientes (nombre, email, telefono) VALUES (%s, %s, %s)"

    try:
        cursor.execute(sql, (nombre, email, telefono))
        conexion.commit()
        print(f"Cliente {nombre} insertado")
    except Error as e:
        print(f"Error: {e}")
    finally:
        cursor.close()
        conexion.close()

def obtener_clientes():
    conexion = conectar()
    cursor = conexion.cursor()

    cursor.execute("SELECT * FROM clientes ORDER BY id DESC")
    clientes = cursor.fetchall()

    cursor.close()
    conexion.close()
    return clientes

# Uso
crear_tabla()
insertar_cliente('Pedro', 'pedro@email.com', '555-1234')
clientes = obtener_clientes()
for cliente in clientes:
    print(cliente)

```

ORM con SQLAlchemy

Un ORM (Object-Relational Mapping) permite trabajar con BD usando objetos Python en lugar de SQL.

Instalación

```
pip install sqlalchemy
```

Concepto Básico

```
from sqlalchemy import create_engine, Column, Integer, String, Float, DateTime
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from datetime import datetime

# Crear base
Base = declarative_base()

# Definir modelo (tabla como clase)
class Usuario(Base):
    __tablename__ = 'usuarios'

    id = Column(Integer, primary_key=True)
    nombre = Column(String(100), nullable=False)
    email = Column(String(100), unique=True, nullable=False)
    edad = Column(Integer)
    fecha_creacion = Column(DateTime, default=datetime.now)

    def __repr__(self):
        return f"<Usuario(nombre='{self.nombre}', email='{self.email}')>"

# Crear engine (conexión)
engine = create_engine('sqlite:///usuarios.db')

# Crear todas las tablas
Base.metadata.create_all(engine)

# Crear sesión para operaciones
Session = sessionmaker(bind=engine)
session = Session()
```

CRUD con ORM

```
# CREATE (Insertar)
nuevo_usuario = Usuario(
    nombre='Juan',
    email='juan@email.com',
    edad=28
)
session.add(nuevo_usuario)
session.commit()

# Múltiples
usuarios = [
    Usuario(nombre='María', email='maria@email.com', edad=25),
    Usuario(nombre='Carlos', email='carlos@email.com', edad=32)
]
```

```

session.add_all(usuarios)
session.commit()

# READ (Leer)
# Todos
todos = session.query(Usuario).all()

# Filtrado
usuario = session.query(Usuario).filter_by(nombre='Juan').first()

# Con condiciones
mayores_25 = session.query(Usuario).filter(Usuario.edad > 25).all()

# UPDATE (Actualizar)
usuario = session.query(Usuario).filter_by(id=1).first()
if usuario:
    usuario.edad = 30
    session.commit()

# DELETE (Eliminar)
usuario = session.query(Usuario).filter_by(id=1).first()
if usuario:
    session.delete(usuario)
    session.commit()

# Cerrar sesión
session.close()

```

Relaciones Entre Tablas

```

from sqlalchemy import ForeignKey, Text
from sqlalchemy.orm import relationship

class Empresa(Base):
    __tablename__ = 'empresas'

    id = Column(Integer, primary_key=True)
    nombre = Column(String(100), nullable=False)
    empleados = relationship('Empleado', back_populates='empresa')

class Empleado(Base):
    __tablename__ = 'empleados'

    id = Column(Integer, primary_key=True)
    nombre = Column(String(100))
    empresa_id = Column(Integer, ForeignKey('empresas.id'))
    empresa = relationship('Empresa', back_populates='empleados')

# Uso
engine = create_engine('sqlite:///empresa.db')
Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
session = Session()

empresa = Empresa(nombre='Tech Corp')
session.add(empresa)
session.commit()

empleado = Empleado(nombre='Ana', empresa_id=empresa.id)
session.add(empleado)
session.commit()

# Acceder desde objeto relacionado

```

```
empresa = session.query(Empresa).first()
print(f"Empleados de {empresa.nombre}: {empresa.empleados}")
```

Buenas Prácticas

1. Usar Variables de Entorno

```
import os
from dotenv import load_dotenv

load_dotenv() # Carga desde .env

DB_HOST = os.getenv('DB_HOST', 'localhost')
DB_USER = os.getenv('DB_USER', 'root')
DB_PASSWORD = os.getenv('DB_PASSWORD')
DB_NAME = os.getenv('DB_NAME')

conexion = mysql.connector.connect(
    host=DB_HOST,
    user=DB_USER,
    password=DB_PASSWORD,
    database=DB_NAME
)
```

Archivo .env:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=miPassword123
DB_NAME=tienda
```

2. Manejo de Excepciones

```
import mysql.connector
from mysql.connector import Error

try:
    conexion = mysql.connector.connect(
        host='localhost',
        user='root',
        password='password',
        database='tienda'
    )
    cursor = conexion.cursor()
    cursor.execute("SELECT * FROM usuarios")

except Error as e:
    print(f"Error en BD: {e}")

except Exception as e:
    print(f"Error inesperado: {e}")

finally:
    if conexion.is_connected():
        cursor.close()
        conexion.close()
```

3. Connection Pooling

```
from mysql.connector import pooling

dbconfig = {
    "host": "localhost",
    "user": "root",
    "password": "password",
    "database": "tienda"
}

pool = pooling.MySQLConnectionPool(
    pool_name="mypool",
    pool_size=5,
    **dbconfig
)

def obtener_conexion():
    return pool.get_connection()

conexion = obtener_conexion()
```

4. Usar Prepared Statements (Prevenir SQL Injection)

```
#  CORRECTO - Usa placeholders
cursor.execute("SELECT * FROM usuarios WHERE id = ?", (1,))

#  INCORRECTO - Concatenación de strings
user_id = request.args.get('id')
cursor.execute(f"SELECT * FROM usuarios WHERE id = {user_id}") # ¡PELIGRO!
```

5. Documentar Esquema

```
# schema.sql
CREATE TABLE usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    edad INTEGER CHECK (edad >= 0),
    fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_email ON usuarios(email);
```

6. Logging

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

logger = logging.getLogger(__name__)

try:
    cursor.execute("INSERT INTO usuarios (nombre) VALUES (?)", ('Juan',))
    conexion.commit()
    logger.info("Usuario insertado correctamente")
except Exception as e:
    logger.error(f"Error al insertar usuario: {e}")
```

Recursos y Referencias

Documentación Oficial

- SQLite: <https://docs.python.org/3/library/sqlite3.html>
- MySQL Connector: <https://dev.mysql.com/doc/connector-python/en/>
- psycopg2: <https://www.psycopg.org/>
- SQLAlchemy: <https://www.sqlalchemy.org/>

Libros Recomendados

- "Database Basics" de José María Amado
- "Learning SQL" de Alan Beaulieu

Comandos SQL Útiles

```
-- Ver estructura de tabla
DESCRIBE usuarios;

-- Ver índices
SHOW INDEXES FROM usuarios;

-- Información de servidor
SELECT VERSION();
```